

ハッカソン2

～マネジメント計画の立て方と設計～

関根 晃太・白木 詩乃

マネジメント計画とは

▶ Grammar

▶ Example

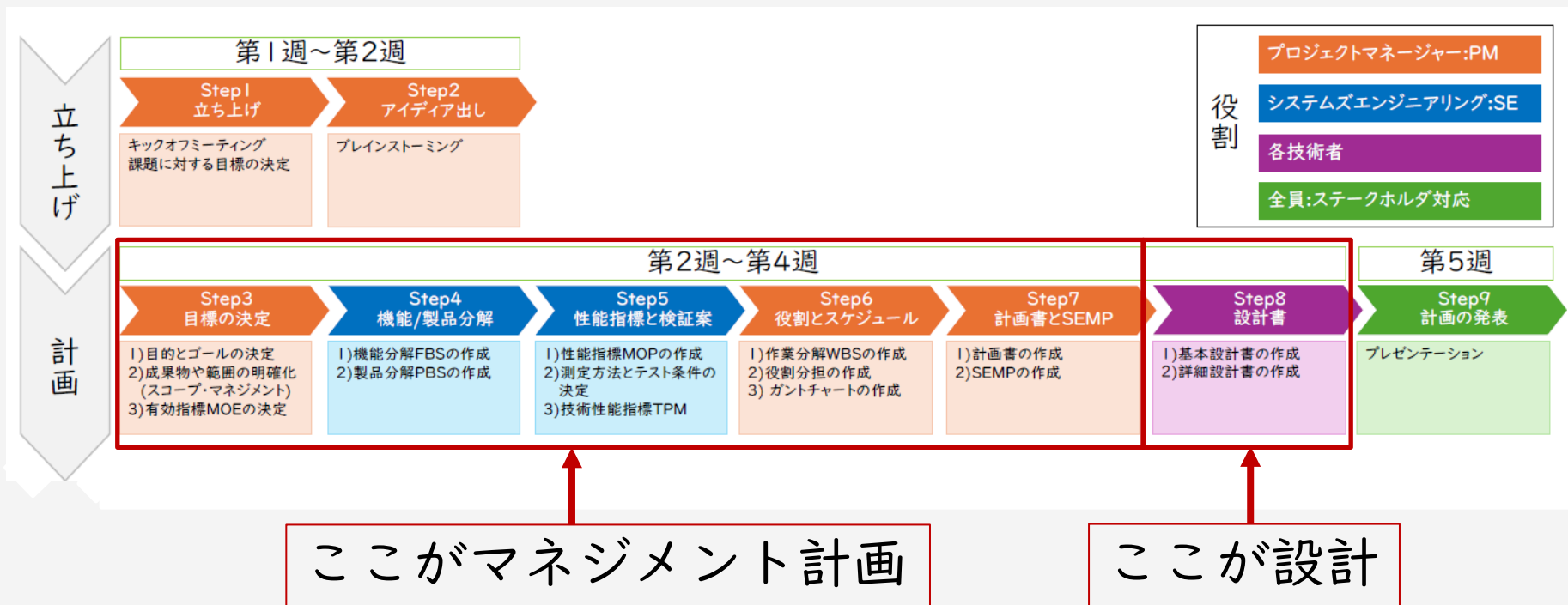
▶ Solution

マネジメント計画とは:

プロジェクトをどのように運用するか, を計画する.

注意: システムの設計とは違う!

(設計はシステムの回路図など組み立て方)



マネジメント計画とは

▶ Grammar

▶ Example

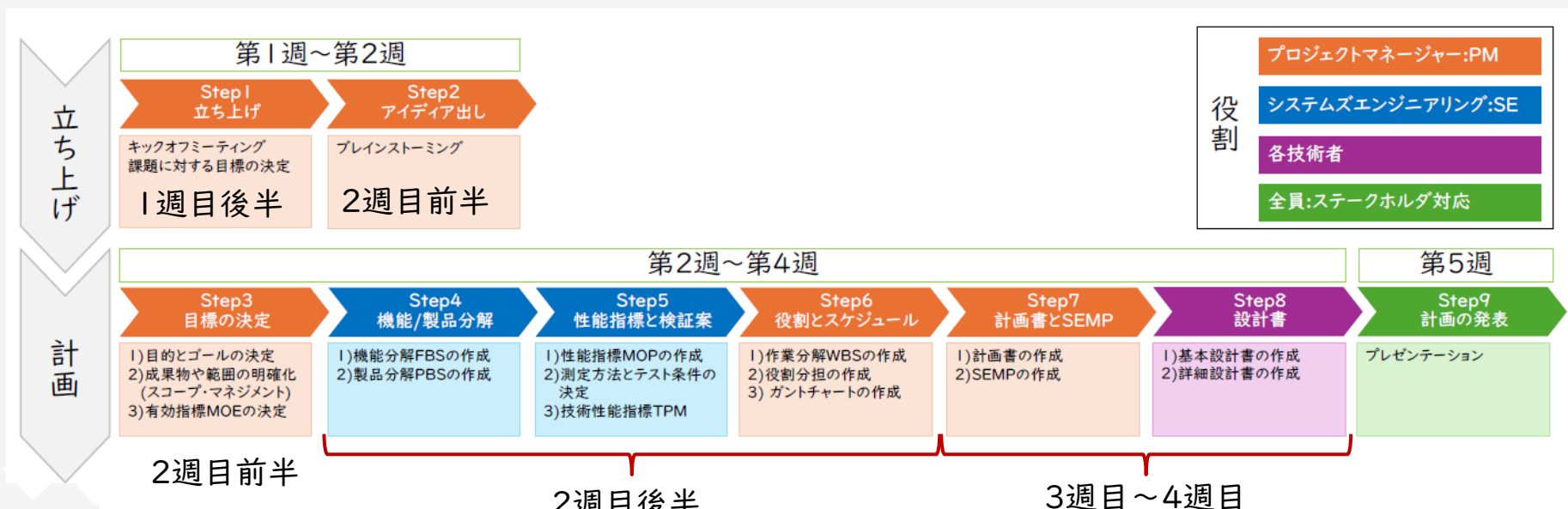
▶ Solution

マネジメント計画とは:

プロジェクトをどのように運用するか，を計画する。

注意: システムの設計とは違う！

(設計はシステムの回路図など組み立て方)



1) 目的とゴール

2) 最終的な成果物，取り扱う範囲，取り扱わない範囲の明確化 (PMのスコープ・マネジメント)

3) ゴールに適した有効指標MOEの決定

MOE:ステークホルダが合格・不合格を決めるための指標

➡ 対応するテスト: 受入テスト



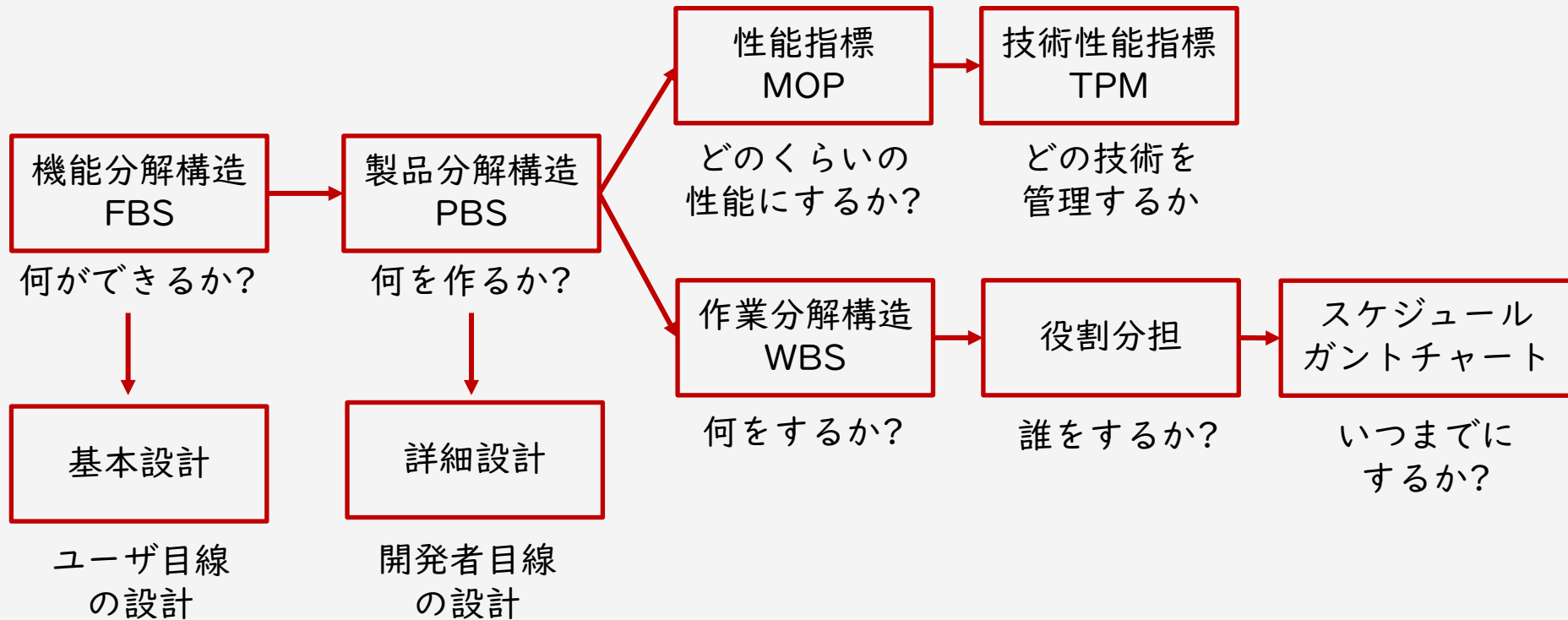
4) 有効指標MOEに対応する性能指標MOPの決定

MOEに対応するMOP:

MOEを達成するために必要となるシステムの性能の指標

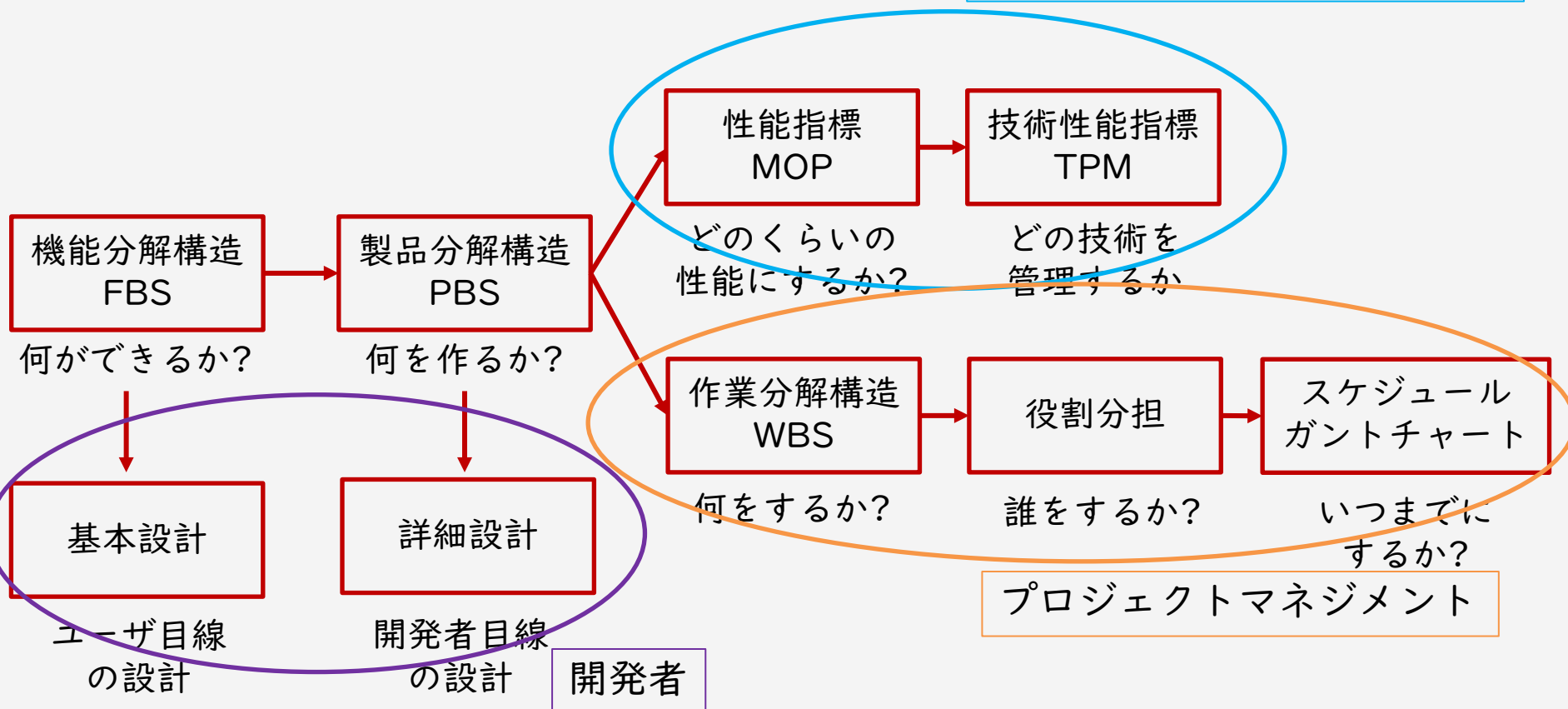
➡ 対応するテスト: システムテスト

目的とゴールが決まったら…

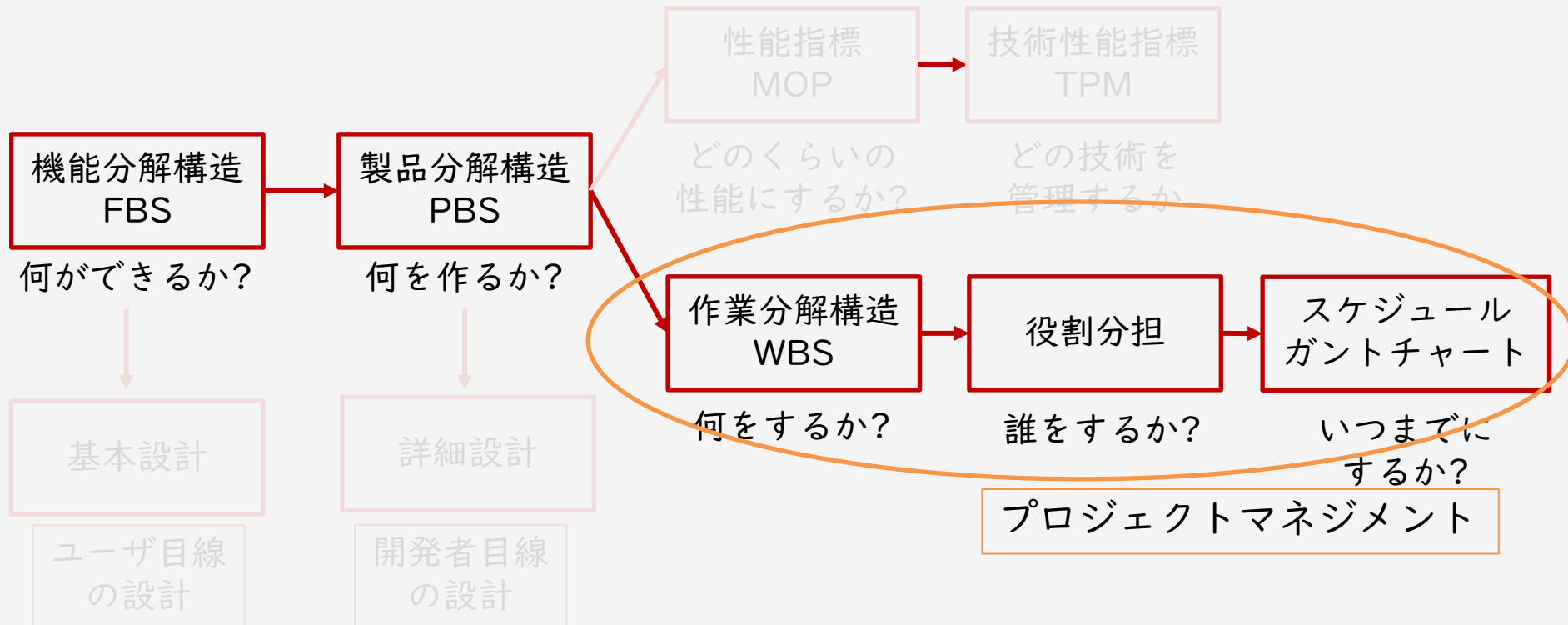


目的とゴールが決まったら…

システムズエンジニアリング



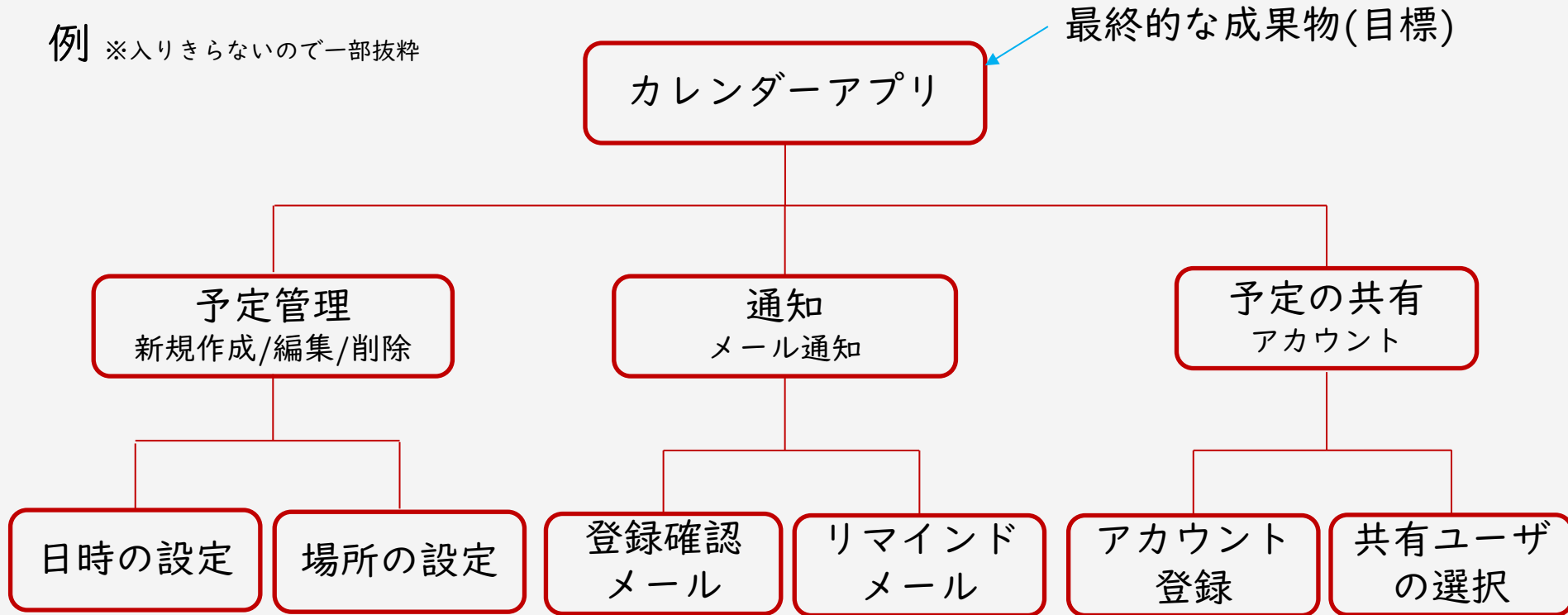
目的とゴールが決まったら…



1. 機能分解構造FBS(Function Breakdown Structure) 最終的な成果物が有するユーザ視点の機能を要素ごとに分解。

例 ※入りきらないので一部抜粋

最終的な成果物(目標)

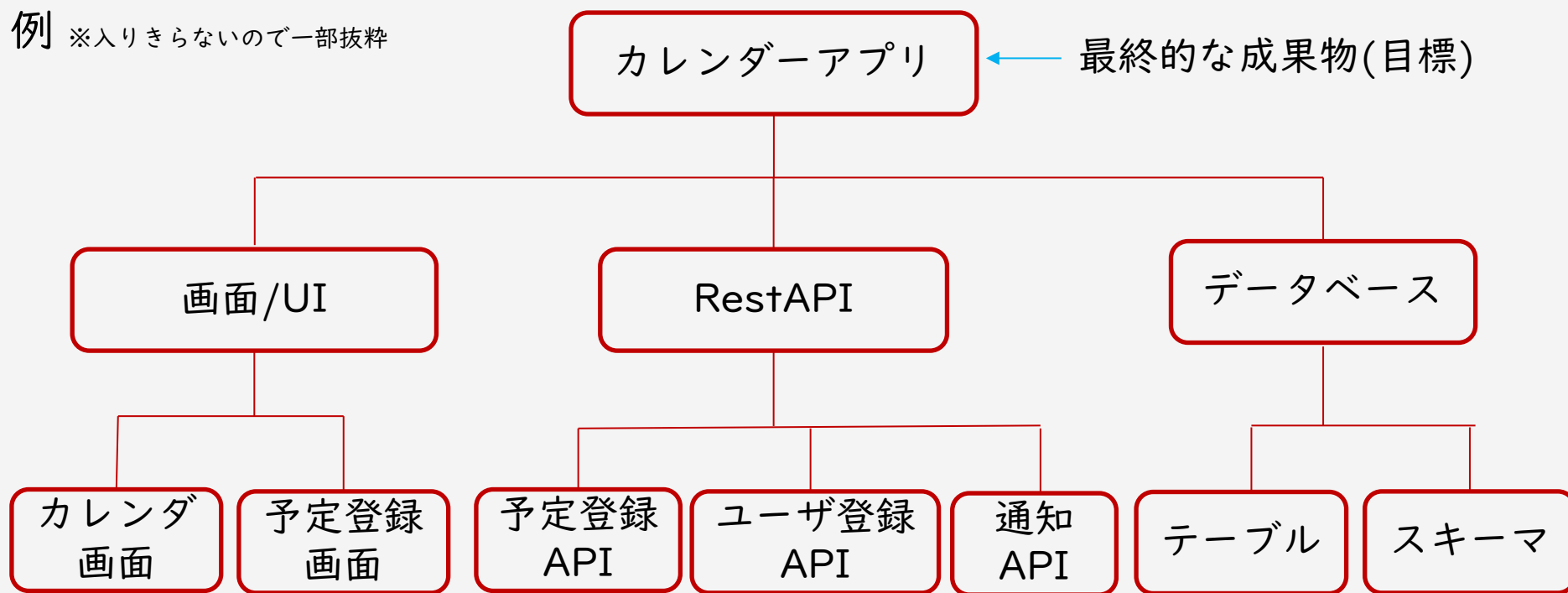


➡ 目標・ゴールをユーザ視点の機能に分解

2. 製品分解構造PBS(Product Breakdown Structure)

何を作るのか?を具体化的にサブ成果物に分解.

例 ※入りきらないので一部抜粋

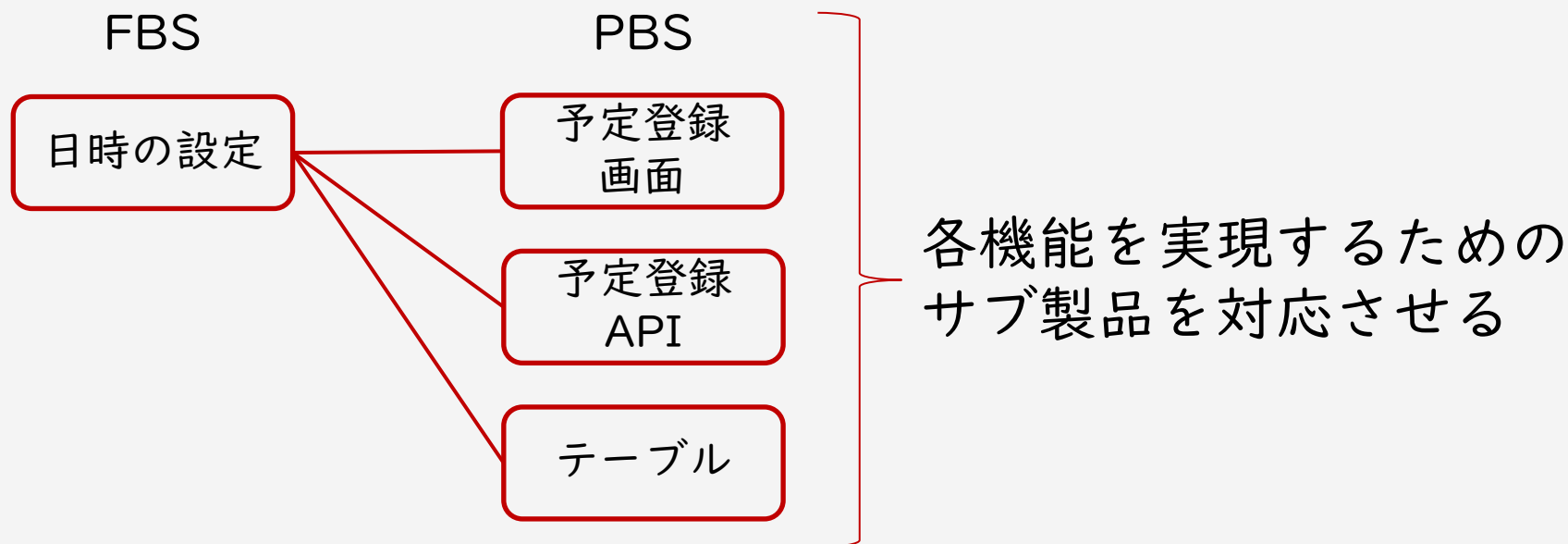


➡ FBSの各機能に対しPBSの各成果物が1つ以上対応

3. FBSとPBSの対応

FBSのすべての機能に対してPBSのサブ成果物が1つ以上を対応.

例 ※入りきらないので一部抜粋



対応が見つからないFBS・PBSの要素は列挙漏れ!!

4. 作業分解構造WBS(Work Breakdown Structure)

PBSのサブ成果物を完成させるために何をするのか作業で分解。

例 ※入りきらないので一部抜粋

カレンダーアプリ

← 最終的な成果物(目標)

設計フェーズ

実装フェーズ

テストフェーズ

基本設計

詳細設計

画面実装

RestAPI
実装

DB
実装

単体
テスト

結合
テスト

カレンダー
デザイン

予定入力
デザイン

カレンダー
画面の実装

...

4. 作業分解構造WBS(Work Breakdown Structure)

PBSのサブ成果物を完成させるために何をするのか作業で分解。

例 ※入りきらないので一部抜粋

0層目: 目標

カレンダーアプリ

← 最終的な成果物(目標)

1層目: 時間的なフェーズ

テストフェーズ

2層目: 要素成果物

画面実装

RestAPI
実装

DB
実装

単体
テスト

結合
テスト

3層目: 活動タスク

カレンダー
画面の実装

...

4. 作業分解構造WBS(Work Breakdown Structure)

PBSのサブ成果物を完成させるために何をするのか作業で分解。

例 ※入りきらないので一部抜粋

0層目: 目標

カレンダーアプリ

← 最終的な成果物(目標)

1層目: 時間的なフェーズ

テストフェーズ

2層目: 要素成果物

画面実装

RestAPI
実装

DB
実装

単体
テスト

結合
テスト

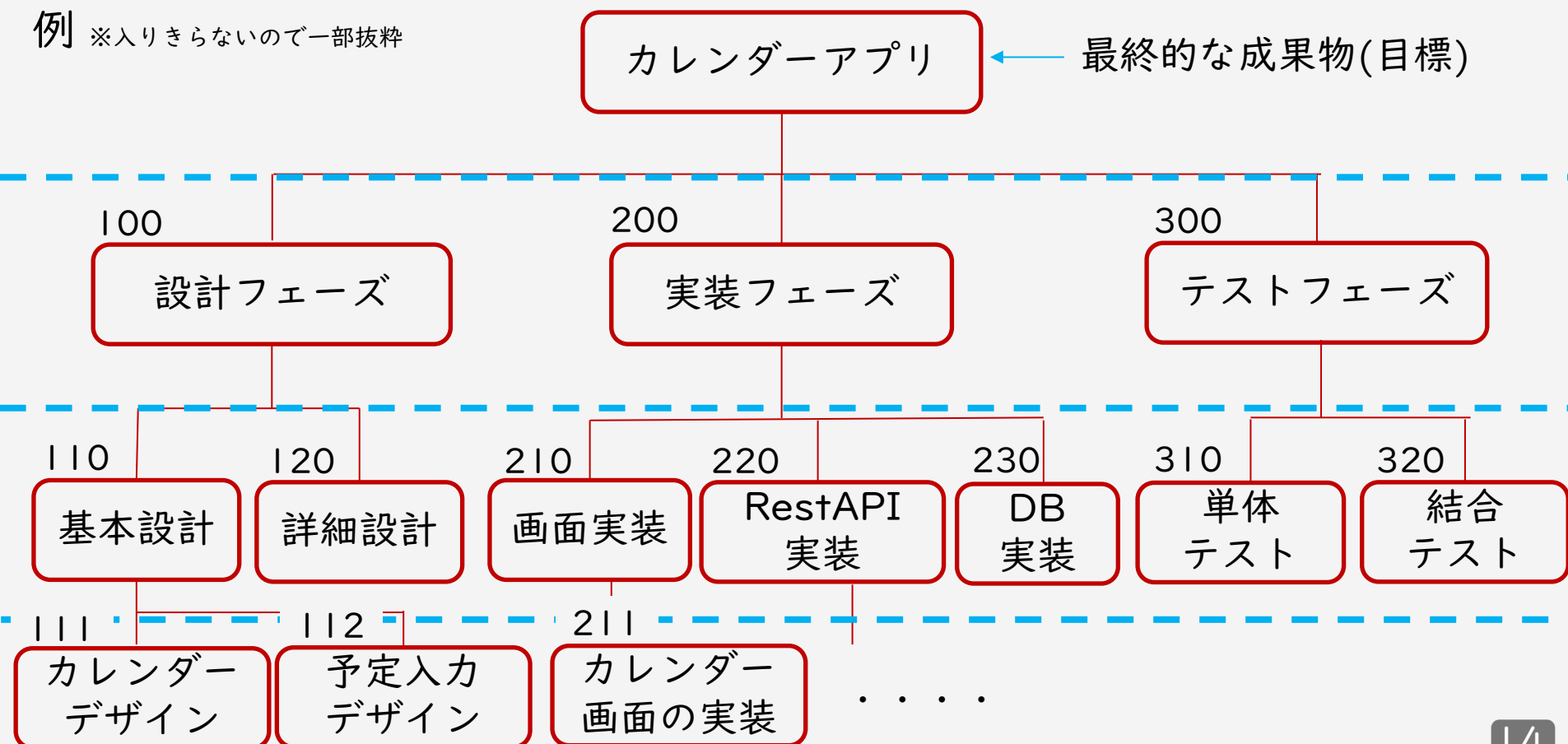
3層目: 活動タスク

活動タスクは8/80ルールを目安に!
8時間(1日)以上80時間(2週間)未満に!!

5. 管理番号

WBSに管理番号を割り当てる.

例 ※入りきらないので一部抜粋

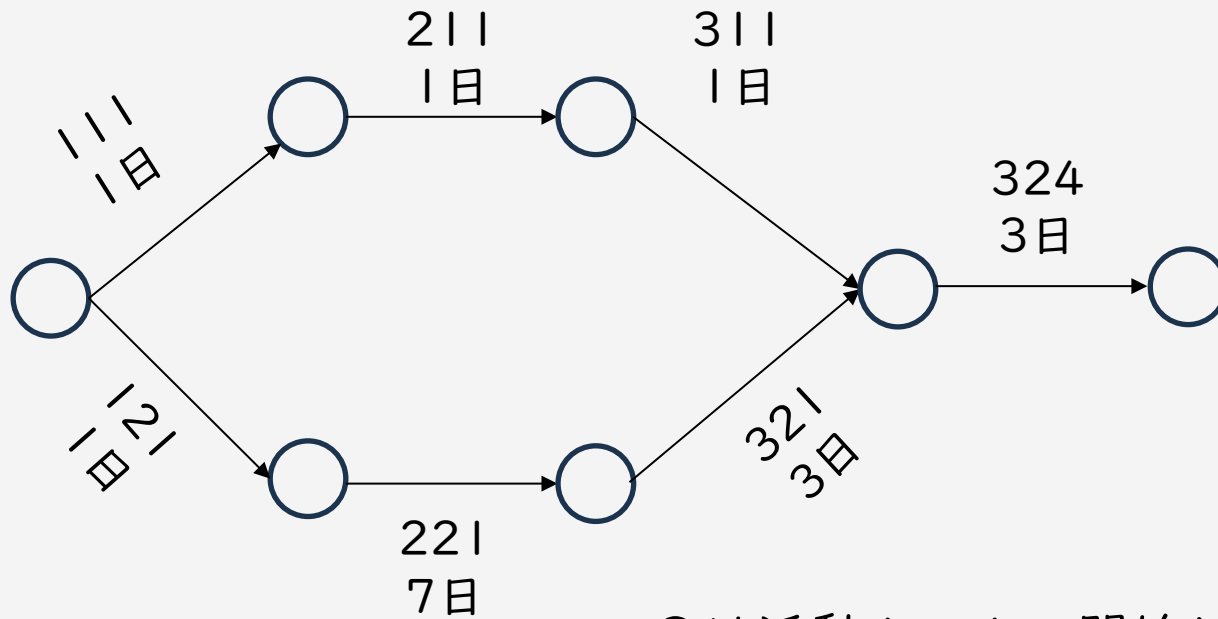


アローダイアグラムとクリティカルパス

6. アローダイアグラムとクリティカルパス

全活動タスクに対し各活動タスクの依存関係を
アローダイアグラムで整理。

活動タスク間の所要時間を見積もり，
活動における最長経路「クリティカルパス」を見つける



○は活動タスクの開始と終了を意味

○と○の間に活動タスクの管理番号と日数

7. 役割分担

WBSの第3層: 活動タスクに対して担当者を割り当てる。特に、アローダイアグラムで見つけたボトルネックを役割分担や割当人数にて解消させる。

WBSを表として表し、各活動タスクに担当者を割り当てる

例 ※入りきらないので一部抜粋F

カレンダーアプリ								
時間的フェーズ		要素成果物	活動タスク	担当者				
100	設計フェーズ	110	基本設計	111	カレンダーデザイン			
				112	予定入力デザイン			
				113	予定詳細画面デザイン			
				114	ユーザ登録デザイン			
				115	ユーザ選択画面デザイン			
		120	詳細設計	121	予定登録CRUD用API URI			
				122	ユーザ登録CRUD用API URI			
				123	通知用API URI			
				124	予定登録CRUD用API関数			
				125	ユーザ登録CRUD用API関数			
				126	通知用API関数			
				127	データベーステーブル設計			
				128	データベーススキーム設計			
				129	データベースカラム設計			
200	実装フェーズ	210	画面実装	211	カレンダー画面の実装			
				212	予定入力画面の実装			
				213	予定詳細画面の実装			
				214	ユーザ登録画面の実装			
				215	ユーザ選択画面の実装			
		220	API実装	221	予定登録CRUD用APIの実装			
				222	ユーザ登録CRUD用APIの実装			
				223	通知用APIの実装			
		230	データベース実装	231	データベースの実装			
				232	テストデータ作成			
		300	テストフェーズ	310	単体テスト	311	カレンダー画面テスト	
						312	予定入力画面画面テスト	
						313	予定詳細画面テスト	
						314	ユーザ登録画面テスト	
315	ユーザ選択画面テスト							
316	予定登録CRUD用APIテスト							
317	ユーザ登録CRUD用APIテスト							
318	通知用APIテスト							
319	データベーステスト							
320	結合テスト			321	予定登録APIとデータベースの接続テスト			
				322	ユーザ登録APIとデータベースの接続テスト			
				323	通知用APIとデータベースの接続テスト			
				324	予定入力画面と予定登録APIの接続テスト			
				325	予定入力画面と通知用APIの接続テスト			
330	システムテスト			331	運用サーバーにおける実環境テスト			
				332	バックアップテスト			
				333	修正デプロイテスト			

スケジュールとガントチャート

8. スケジュールとガントチャート

プロジェクトの進捗状況を把握するためのガントチャートを作成

例 ※入りきらないので一部抜粋F

カレンダーアプリ				4月								5月													
時間的フェーズ	要素成果物	活動タスク	担当者	23日	24日	25日	26日	27日	28日	29日	30日	1日	2日	3日	4日	5日	6日	7日	8日	9日	10日	11日	12日	13日	14日
100	設計フェーズ	110 基本設計	111 カレンダーデザイン																						
			112 予定入力デザイン																						
			113 予定詳細画面デザイン																						
			114 ユーザ登録デザイン																						
			115 ユーザ選択画面デザイン																						
		120 詳細設計	121 予定登録CRUD用API URI																						
			122 ユーザ登録CRUD用API URI																						
			123 通知用API URI																						
			124 予定登録CRUD用API関数																						
			125 ユーザ登録CRUD用API関数																						
			126 通知用API関数																						
			127 データベーステーブル設計																						
			128 データベーススキーム設計																						
			129 データベースカラム設計																						
200	実装フェーズ	210 画面実装	211 カレンダー画面の実装																						
			212 予定入力画面の実装																						
			213 予定詳細画面の実装																						
			214 ユーザ登録画面の実装																						
			215 ユーザ選択画面の実装																						
		220 API実装	221 予定登録CRUD用APIの実装																						
			222 ユーザ登録CRUD用APIの実装																						
			223 通知用APIの実装																						
		230 データベース実装	231 データベースの実装																						
			232 テストデータ作成																						

粒度は8/80ルールに則り1日単位

スケジュールとガントチャート

8. スケジュールとガントチャート

プロジェクトの進捗状況を把握するためのガントチャートを作成

例 ※入りきらないので一部抜粋

カレンダーアプリ				4月										5月														
時間的フェーズ		要素成果物		活動タスク	担当者	23日	24日	25日	26日	27日	28日	29日	30日	1日	2日	3日	4日	5日	6日	7日	8日	9日	10日	11日	12日	13日	14日	
100	設計フェーズ	110	基本設計	111 カレンダーデザイン																								
				112 予定入力デザイン																								
				113 予定詳細画面デザイン																								
				114 ユーザ登録デザイン																								
				115 ユーザ選択画面デザイン																								
		120	詳細設計	121 予定登録CRUD用API URI																								
				122 ユーザ登録CRUD用API URI																								
				123 通知用API URI																								
				124 予定登録CRUD用API関数																								
				125 ユーザ登録CRUD用API関数																								
				126 通知用API関数																								
				127 データベーステーブル設計																								
				128 データベーススキーム設計																								
				129 データベースカラム設計																								
200	実装フェーズ	210	画面実装	211 カレンダー画面の実装																								
				212 予定入力画面の実装																								
				213 予定詳細画面の実装																								
				214 ユーザ登録画面の実装																								
				215 ユーザ選択画面の実装																								
		220	API実装	221 予定登録CRUD用APIの実装																								
				222 ユーザ登録CRUD用APIの実装																								
				223 通知用APIの実装																								
230	データベース実装	231 データベースの実装																										
		232 テストデータ作成																										

- 1) 他科目の試験や休日などのプロジェクト外の予定も加味
- 2) プロジェクト全体の終盤にバッファを設ける(各項目に設けずに)
- 3) 単体テスト, 結合テスト, システムテストなどのテストも含める

スケジュールとガントチャート

8. スケジュールとガントチャート

プロジェクトの進捗状況を把握するためのガントチャートを作成

例 ※入りきらないので一部抜粋

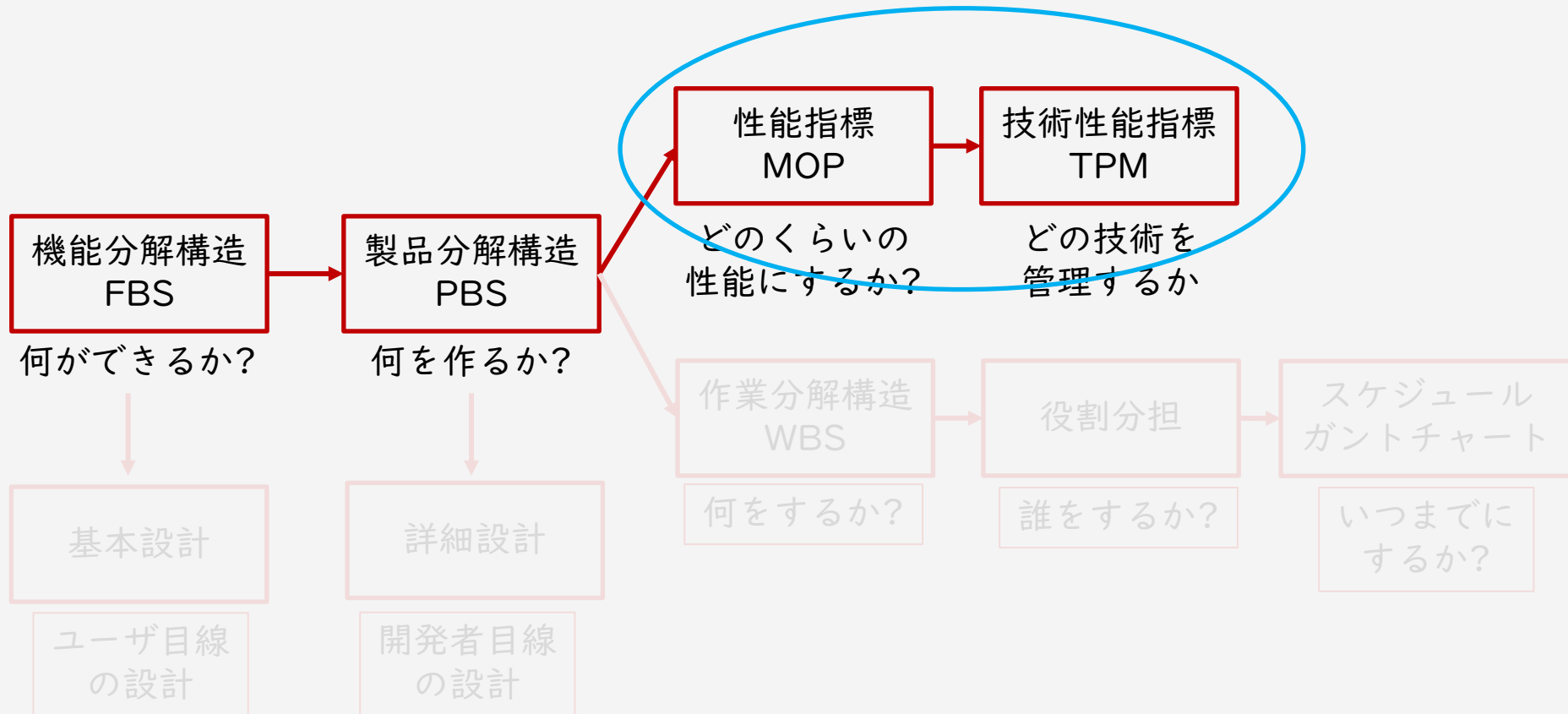
カレンダーアプリ				4月							5月																				
時間的フェーズ		要素成果物		活動タスク		担当者		23日	24日	25日	26日	27日	28日	29日	30日	1日	2日	3日	4日	5日	6日	7日	8日	9日	10日	11日	12日	13日	14日		
100	設計フェーズ	110	基本設計	111	カレンダーデザイン																										
				112	予定入力デザイン																										
				113	予定詳細画面デザイン																										
				114	ユーザ登録デザイン																										
				115	ユーザ選択画面デザイン																										
		120	詳細設計	121	予定登録CRUD用API URI																										
				122	ユーザ登録CRUD用API URI																										
				123	通知用API URI																										
				124	予定登録CRUD用API関数																										
				125	ユーザ登録CRUD用API関数																										
				126	通知用API関数																										
				127	データベーステーブル設計																										
				128	データベーススキーム設計																										
				129	データベースカラム設計																										
200	実装フェーズ	210	画面実装	211	カレンダー画面の実装																										
				212	予定入力画面の実装																										
				213	予定詳細画面の実装																										
				214	ユーザ登録画面の実装																										
				215	ユーザ選択画面の実装																										
		220	API実装	221	予定登録CRUD用APIの実装																										
				222	ユーザ登録CRUD用APIの実装																										
				223	通知用APIの実装																										
		230	データベース実装	231	データベースの実装																										
				232	テストデータ作成																										

プロジェクト実行中にガントチャートで進捗を監視!

プロジェクトに変更があった場合, その都度, 修正する

目的とゴールが決まったら…

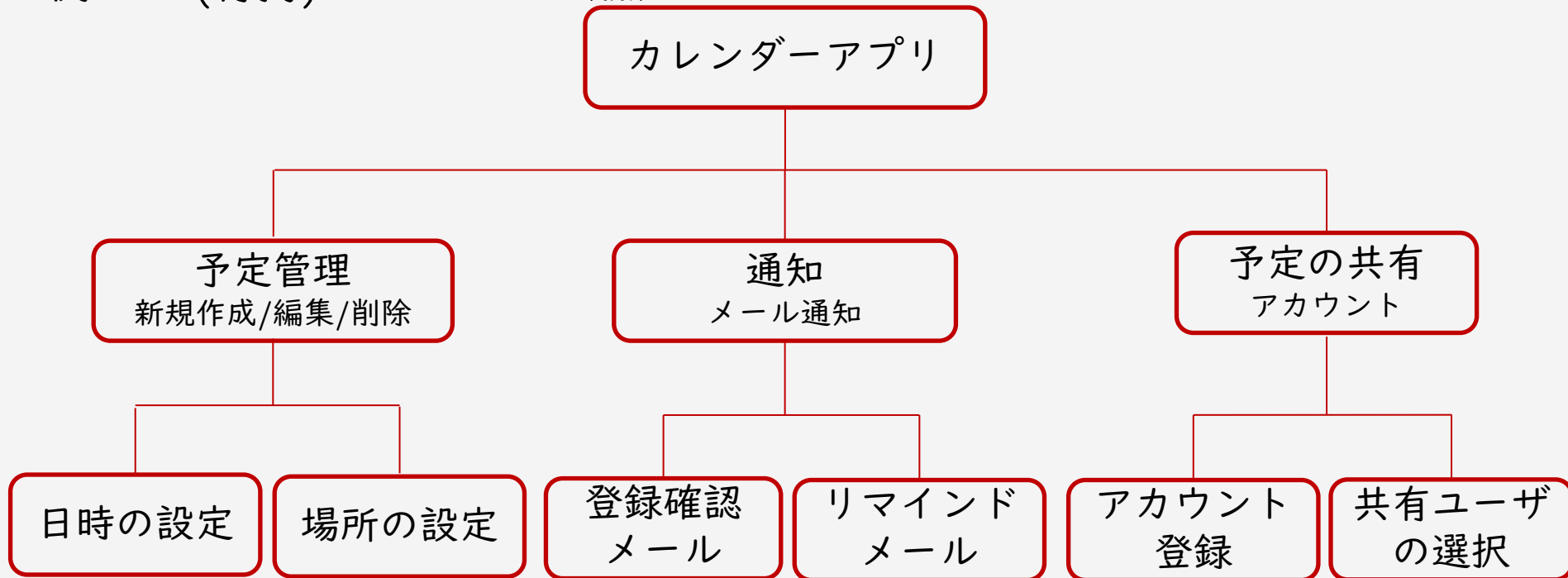
システムズエンジニアリング



1. 性能指標MOP(Measure of Performance)

機能分解構造FBSの各要素ごとに1つ以上のMOPを設定する

例 FBS(再掲) ※入りきらないので一部抜粋



I. 性能指標MOP(Measure of Performance)

機能分解構造FBSの各要素ごとに1つ以上のMOPを設定する

例 FBS(再掲) ※入りきらないので一部抜粋

システムテスト

カレンダーアプリ

結合テスト

予定管理
新規作成/編集/削除

通知
メール通知

予定の共有
アカウント

単体テスト

日時の設定

場所の設定

登録確認
メール

リマインド
メール

アカウント
登録

共有ユーザ
の選択

➡ 各要素に設定したMOPをそれぞれのテストで確認!

2. 技術性能指標TPM(Technical Performance Measure) システムの最終的な性能に関わる技術の管理方法の決定

例 ※入りきらないので一部抜粋

日時の設定 : カレンダーアプリにおいて予定日時の設定は最重要

リマインド
メール : リマインド日時の誤りはシステムの根幹にかかわる

アカウント
登録 共有ユーザ
の選択 : 個人情報の観点からセキュリティは重要



プロジェクト実行中に性能をどのような数値で管理するか決める

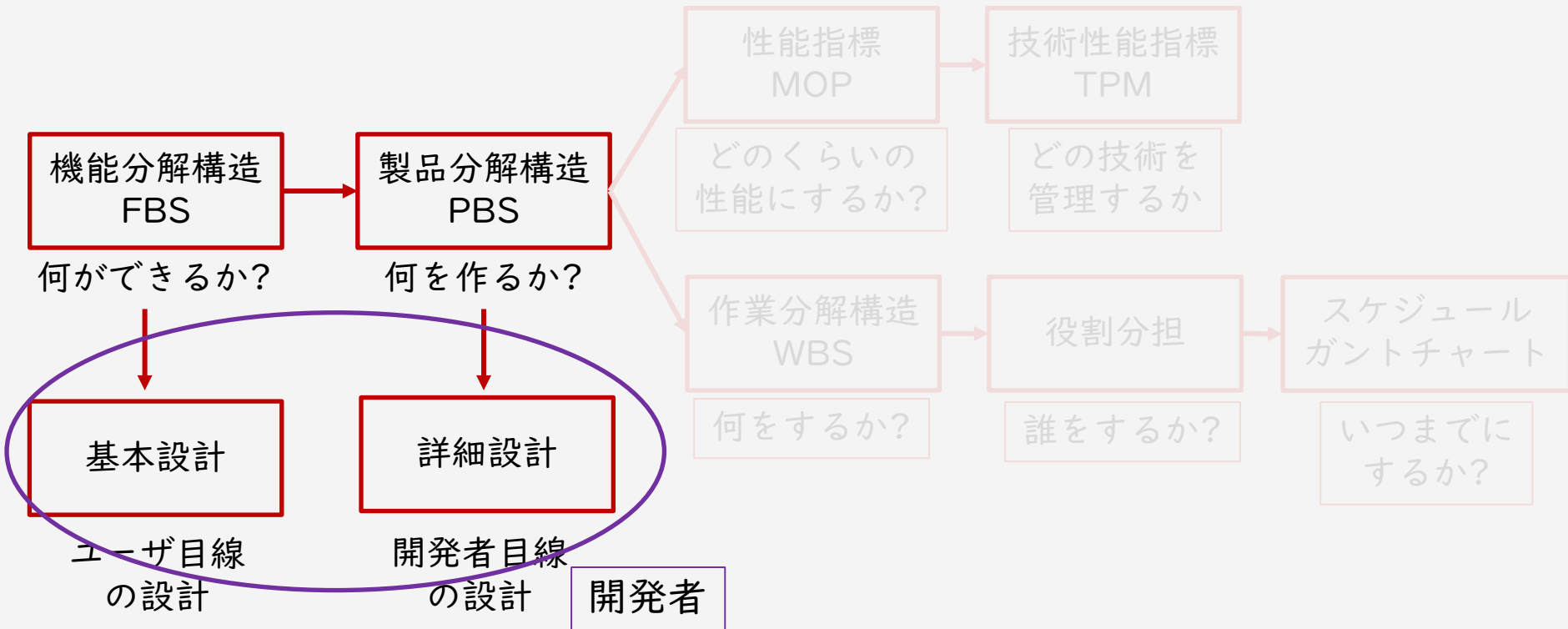
3. (おまけ)技術成熟度レベルTRL

機能分解構造FBSの各要素ごとに対して、現在どのレベルにいるか、進捗を確認する指標

TRL	定義
1	概念レベル: アイディアや原理が確立されており、説明できるレベル
2	基本動作レベル: 単体で動作確認が可能なレベル
3	結合動作レベル: 機能を結合し、実際の使用環境に近い環境で動作確認が可能なレベル
4	実運用レベル: 実際の使用環境で安定的な稼働が確認できるレベル

※プロジェクトの実行中にTRLでFBSの各要素の進捗を確認するもので、ここでは作成することはありません。

目的とゴールが決まったら…



1. 基本設計

機能分解構造FBSの各要素に対応したユーザ目線の設計.

ユーザが

- 1) どのように操作するか
 - 2) どのような画面を見るのか
- などを設計する.

ステークホルダやユーザに確認してもらう.

例) 基本設計の記載例:

- ・システム構成図
- ・画面/ボタナー覧(表示する文字やボタン配置も含む)
- ・状態遷移図(ボタン操作などによる画面などの状態の遷移)
- ・ユーザが利用するAPIやボタンの仕様書

※開発者のみが利用するAPIやボタンは含まない

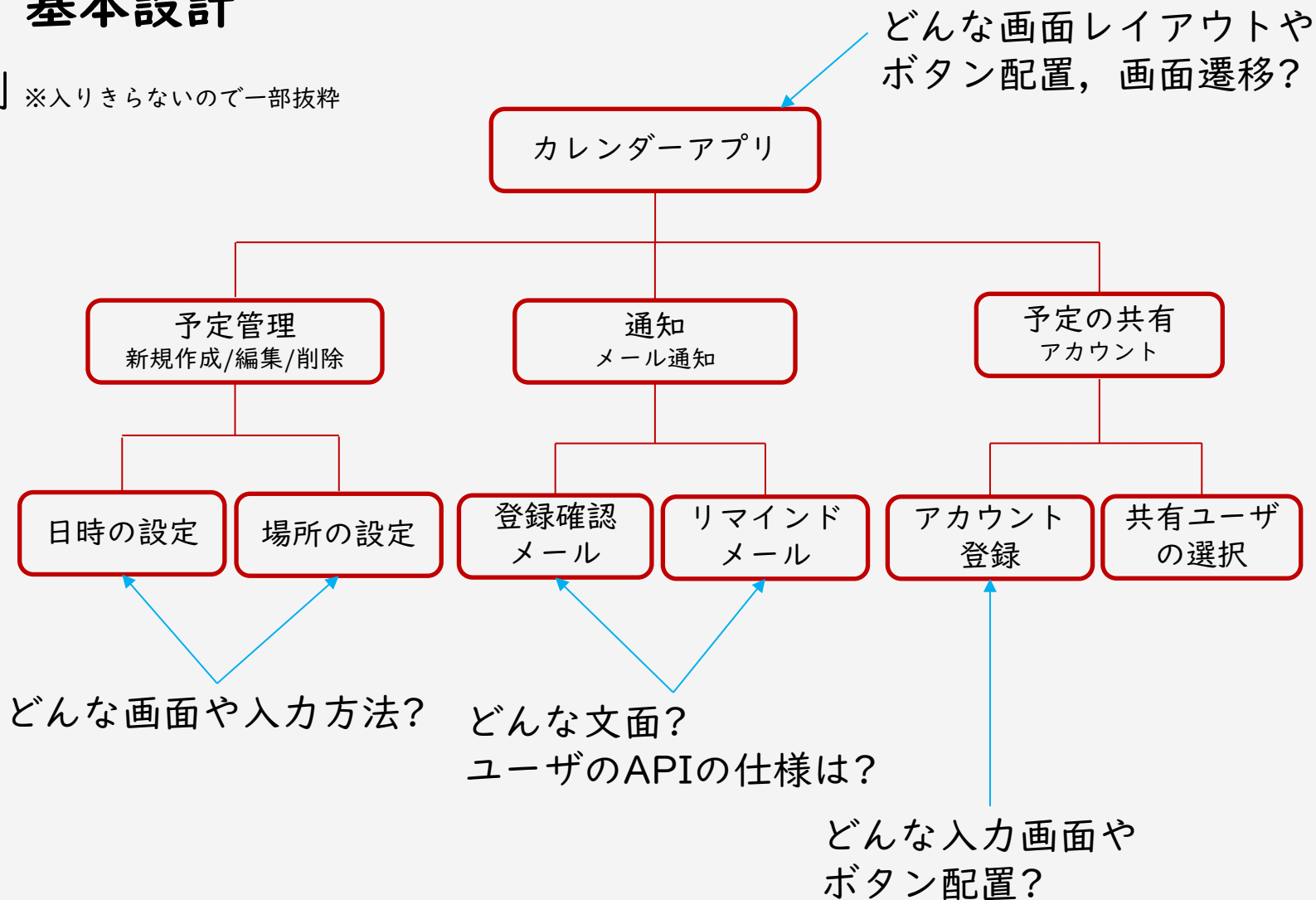


ユーザが触れる部分に関する設計書!!

ユーザが使い方やどのようなシステムかわかるように書く!

1. 基本設計

例 ※入りきらないので一部抜粋



1. 基本設計

カレンダー: メイン画面 A-1

矢印をクリックすると
表示される月が変化

カレンダーアプリ

← 20xx年 4月 →						
月	火	水	木	金	土	日
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			



日付をクリックすると
詳細画面(B-1)へ遷移

当日は色分けされる

自身のユーザ詳細画面
(D-1)へ遷移

未登録時はユーザ登録画面(E-1)へ

予定追加ボタン

クリックすると予定追加画面(C-1)へ遷移

カレンダー共有機能ボタン
共有されるユーザ選択画面へ
(F-1)遷移

2. 詳細設計

製品分解構造PBSに基づいて実際に実装する内部構造の設計図。
開発者が

- 1) 何を実装するのか
 - 2) どのように実装するのか
- などを設計する。

例) 詳細設計の記載例:

- ・ 回路図などのシステムの設計図
- ・ 処理の流れを表すシーケンス図やフローチャート
- ・ クラス(構造体と付随する関数)同士の関係図(クラス図)
- ・ 内部で使用する関数やAPIの仕様



設計チームと実装チームが別でも
詳細設計を見て実装できるようにする!

Schedule

-int id: スケジュールid
-Date startdate: 開始時刻
-Date enddate: 終了時刻
-String content: 予定の内容
-Bool flag: 完了済みチェック

+create(sd, ed, cnt) : id 予定作成
+read(id) : content 予定読取
+update(id, sd, ed, cnt) : bool 予定修正
+delete(id) : bool 予定削除
+check_flag(id) : bool 完了済みか確認

User

-int id: ユーザーid
-String name: 氏名
-String mail: メールアドレス
-String pass: パスワード

+create(nm,ml,ps) : id ユーザ作成
+search(name) : id 氏名検索
+update(id, nm,ml,ps) : bool ユーザ修正
+delete(id) : bool ユーザ削除

Calendar

-User user
-list<Schedule> schedules

+read() : 全スケジュールを取出
+add_schedule(Schedule sch)
+update_schedule(Schedule sch)
+delete_schedule(Schedule sch)
+add_user(User user)
+update_user(User user)
+delete_user(User user)

Schedule

-int id: スケジュールid
-Date startdate: 開始時刻
-Date enddate: 終了時刻
-String content: 予定の内容
-Bool flag: 完了済みチェック

すべてのメソッド関数の仕様を書く

+create(sd, ed, cnt) : id 予定作成
+read(id) : content 予定読取
+update(id, sd, ed, cnt) : bool 予定修正
+delete(id) : bool 予定削除
+check_flag(id) : bool 完了済みか確認

User

-int id: ユーザーid
-String name: 氏名
-String mail: メールアドレス
-String pass: パスワード

+create(nm,ml,ps) : id ユーザ作成
+search(name) : id 氏名検索
+update(id, nm,ml,ps) : bool ユーザ修正
+delete(id) : bool ユーザ削除

Calendar

-User user
-list<Schedule> schedules

+read() : 全スケジュールを取出
+add_schedule(Schedule sch)
+update_schedule(Schedule sch)
+delete_schedule(Schedule sch)
+add_user(User user)
+update_user(User user)
+delete_user(User user)

関数の仕様

```
int Schedule::create( Date sd, Date ed, String cnt)
```

引数:

Date sd : 開始日

Date ed : 終了日

String cnt : 予定の内容

戻り値:

int id : スケジュールID

内容:

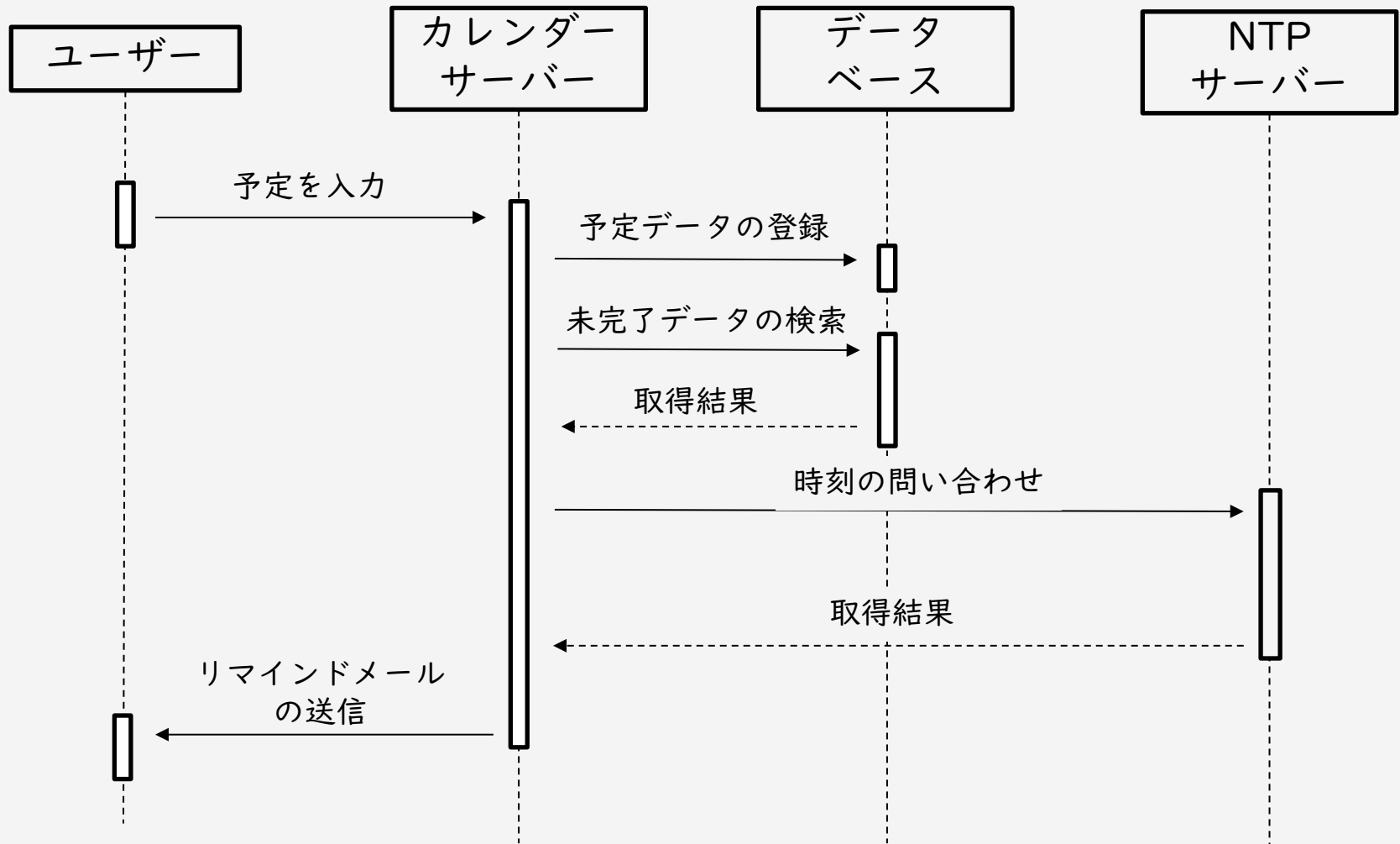
Scheduleオブジェクトのインスタンスに予定を登録する



すべての関数で作成.

必要に応じてエラーコードやフラグの意味も記載する

リマインドメールの仕組み



計画と設計のレビュー

レビュー

計画や設計は一人で作成していると記載漏れや勘違いが発生。

計画書や設計書を作成したら…

レビューの役割分担: 誰がどの部分を確認するか

レビューの方法:

1) チェックリスト形式, 文書形式.

※口頭のみは言った言わないが発生するため不可

(口頭で行う場合, 事前に準備してから口頭でも伝える)

2) どこをどのように修正するのか, 明確にする

 他人が読んでも追加説明なく「プロジェクトの実行」及び「システムの実装」ができるか客観的に確認